

Competitive Programming

Section 8 Editorial: Binary search on the answer

Competitive Programming Course

2026-07-16

Table of contents

1	How to read this editorial	4
2	Problem A: Square Threshold	5
2.1	Restatement	5
2.2	Important constraints and traps	5
2.3	Observations	5
2.4	Algorithm	5
2.5	Correctness	6
2.6	Complexity	6
2.7	C++ solution	6
2.8	Python solution	7
2.9	Common wrong answers	7
3	Problem B: Maximum Minimum Distance	8
3.1	Restatement	8
3.2	Important constraints and traps	8
3.3	Observations	8
3.4	Why the greedy check works	8
3.5	Algorithm	8
3.6	Correctness	9
3.7	Complexity	9
3.8	C++ solution	9
3.9	Python solution	10
3.10	Common wrong answers	11
4	Problem C: Minimum Capacity	12
4.1	Restatement	12
4.2	Important constraints and traps	12
4.3	Observations	12
4.4	Algorithm	12
4.5	Correctness	13
4.6	Complexity	13
4.7	C++ solution	13
4.8	Python solution	14
4.9	Common wrong answers	15
5	Stress testing strategy	16
6	What to carry forward	17
7	Problem D: Factory Deadline	18
7.1	Restatement and predicate	18
7.2	Bounds, trace, and algorithm	18
7.3	Correctness proof	18
7.4	Complexity and implementation notes	18

7.5	Full C++ solution	19
7.6	Full Python solution	20

1 How to read this editorial

For each problem, separate the work into two parts:

1. the predicate;
2. the binary search template.

A correct predicate with wrong bounds fails. Correct bounds with a wrong predicate also fail. Treat both as proof obligations.

2 Problem A: Square Threshold

2.1 Restatement

Given n , find the smallest integer $x \geq 0$ such that:

```
x*x >= n
```

This is the ceiling of the square root of n .

2.2 Important constraints and traps

n can be 10^{18} , so x can be 10^9 . In C++, compute $x*x$ using `__int128` to avoid overflow in similar variants and to make the comparison unambiguous.

The case $n = 0$ should return 0.

2.3 Observations

Define:

```
feasible(x) = (x*x >= n)
```

If x is feasible, every larger integer is feasible. The predicate has shape:

```
False False ... True True
```

We need the first true.

2.4 Algorithm

```
lo = 0
hi = 1000000000
while lo < hi:
    mid = (lo + hi) // 2
    if mid*mid >= n:
        hi = mid
    else:
        lo = mid + 1
print lo
```

2.5 Correctness

The search maintains that the answer lies in $[lo, hi]$. Initially this is true because the answer is nonnegative and at most 10^9 .

At each step, let mid be inside the interval. If $mid * mid \geq n$, then mid is feasible, and the smallest feasible value is at most mid , so it remains in $[lo, mid]$. Setting $hi = mid$ preserves the invariant. Otherwise mid is not feasible, and no value $\leq mid$ can be feasible because the predicate is monotone. The answer lies in $[mid + 1, hi]$, so setting $lo = mid + 1$ preserves the invariant.

The loop stops when $lo = hi$. The invariant says the answer is in this single-value interval, so lo is the required answer.

2.6 Complexity

The search range has size $10^9 + 1$, so there are about 30 iterations. Each iteration is $O(1)$.

2.7 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
using i128 = __int128_t;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int64 n;
        cin >> n;
        int64 lo = 0;
        int64 hi = 1'000'000'000LL;
        while (lo < hi) {
            int64 mid = lo + (hi - lo) / 2;
            if ((i128)mid * mid >= n) {
                hi = mid;
            } else {
                lo = mid + 1;
            }
        }
        cout << lo << '\n';
    }
    return 0;
}
```

2.8 Python solution

```
import sys

def ceil_sqrt(n: int) -> int:
    lo = 0
    hi = 1_000_000_000
    while lo < hi:
        mid = (lo + hi) // 2
        if mid * mid >= n:
            hi = mid
        else:
            lo = mid + 1
    return lo

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
out = [str(ceil_sqrt(data[i])) for i in range(1, t + 1)]
sys.stdout.write("\n".join(out))
```

2.9 Common wrong answers

- Returning floor square root instead of ceiling square root.
- Mishandling $n = 0$.
- Using floating-point `sqrt` and rounding incorrectly near large squares.
- Overflowing `mid * mid` in C++ variants with larger bounds.

3 Problem B: Maximum Minimum Distance

3.1 Restatement

Choose k positions from n line positions to maximize the minimum distance between chosen positions.

3.2 Important constraints and traps

Positions are supplied in non-decreasing order. Duplicates may exist, making answer 0 possible.

This is a maximum feasible problem:

```
can place with distance d?
```

If distance d is feasible, every smaller distance is also feasible.

3.3 Observations

For a fixed distance d , greedily choose the leftmost possible positions:

```
choose first position  
then choose the next position at least d away
```

If this greedy process places at least k positions, then d is feasible.

3.4 Why the greedy check works

Choosing an earlier valid position cannot make future choices harder. It leaves at least as much room to the right as choosing a later valid position. Therefore the greedy check maximizes the number of positions placed for a fixed d .

3.5 Algorithm

```

lo = 0                # feasible
hi = max_position - min_position + 1  # infeasible
while lo + 1 < hi:
    mid = (lo + hi) // 2
    if can_place(mid):
        lo = mid
    else:
        hi = mid
print lo

```

3.6 Correctness

For any distance d , the greedy check returns true exactly when k positions can be chosen with pairwise adjacent chosen positions at least d apart. The greedy argument above shows that if greedy cannot place k , no other method can.

The predicate is monotone: if distance d is feasible, then every smaller distance is feasible using the same chosen positions. Thus the truth pattern is `True...False`, and the binary search maintains `lo` feasible and `hi` infeasible. When the loop terminates, no integer lies between `lo` and `hi`, so `lo` is the largest feasible distance.

3.7 Complexity

Each feasibility check is $O(n)$, and there are $O(\log \text{ range})$ checks. Memory is $O(1)$ besides the array.

3.8 C++ solution

```

#include <bits/stdc++.h>
using namespace std;

bool can_place(const vector<long long> &pos, int k, long long dist) {
    int count = 1;
    long long last = pos[0];
    for (int i = 1; i < (int)pos.size(); ++i) {
        if (pos[i] - last >= dist) {
            ++count;
            last = pos[i];
            if (count >= k) return true;
        }
    }
    return false;
}

int main() {
    ios::sync_with_stdio(false);

```

```

cin.tie(nullptr);

int T;
cin >> T;
while (T--) {
    int n, k;
    cin >> n >> k;
    vector<long long> pos(n);
    for (long long &x : pos) cin >> x;
    long long lo = 0;
    long long hi = pos.back() - pos.front() + 1;
    while (lo + 1 < hi) {
        long long mid = lo + (hi - lo) / 2;
        if (can_place(pos, k, mid)) lo = mid;
        else hi = mid;
    }
    cout << lo << '\n';
}
return 0;
}

```

3.9 Python solution

```

import sys

def can_place(pos: list[int], k: int, dist: int) -> bool:
    count = 1
    last = pos[0]
    for x in pos[1:]:
        if x - last >= dist:
            count += 1
            last = x
            if count >= k:
                return True
    return False

def solve_case(pos: list[int], k: int) -> int:
    lo = 0
    hi = pos[-1] - pos[0] + 1
    while lo + 1 < hi:
        mid = (lo + hi) // 2
        if can_place(pos, k, mid):
            lo = mid
        else:
            hi = mid
    return lo

```

```

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    k = data[idx + 1]
    idx += 2
    pos = data[idx:idx + n]
    idx += n
    out.append(str(solve_case(pos, k)))
sys.stdout.write("\n".join(out))

```

3.10 Common wrong answers

- Binary searching the first false instead of the last true and printing the wrong boundary.
- Treating equal positions as if they had a positive distance.
- Using a non-greedy or quadratic feasibility check.
- Assuming positions are distinct.

4 Problem C: Minimum Capacity

4.1 Restatement

Ship packages in order within at most d days. Each day ships a contiguous prefix of remaining packages whose total weight is at most the capacity. Find the minimum feasible capacity.

4.2 Important constraints and traps

The capacity must be at least the heaviest package. It is always sufficient to use capacity equal to the sum of all weights.

This is a minimum feasible problem:

```
can ship with capacity C?
```

If capacity C works, every larger capacity also works.

4.3 Observations

For a fixed capacity, greedily fill each day until adding the next package would exceed capacity. This minimizes the number of days for that capacity.

Then check whether the number of days is at most d .

4.4 Algorithm

```
lo = max(weights)
hi = sum(weights)
while lo < hi:
    mid = (lo + hi) // 2
    if days_needed(mid) <= d:
        hi = mid
    else:
        lo = mid + 1
print lo
```

4.5 Correctness

For a fixed capacity, the greedy day-count scan ships as many packages as possible each day before starting a new day. Starting a new day earlier cannot allow fewer total days, because it leaves less capacity used on the current day and does not change package order. Therefore the greedy scan computes the minimum possible days for that capacity.

The predicate `days_needed(C) ≤ d` is monotone: increasing capacity cannot increase the number of required days. The binary search maintains that the minimum feasible capacity remains in `[lo, hi]`. When `mid` is feasible, the answer is at most `mid`; otherwise it is larger than `mid`. When the loop ends, `lo = hi` and is exactly the minimum feasible capacity.

4.6 Complexity

Each feasibility check scans the array in $O(n)$. The number of checks is $O(\log \text{sum}(\text{weights}))$. Memory is $O(1)$ besides the input array.

4.7 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int days_needed(const vector<long long> &weights, long long capacity) {
    int days = 1;
    long long cur = 0;
    for (long long w : weights) {
        if (cur + w <= capacity) {
            cur += w;
        } else {
            ++days;
            cur = w;
        }
    }
    return days;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, d;
        cin >> n >> d;
        vector<long long> weights(n);
        long long lo = 0;
        long long hi = 0;
```

```

    for (long long &w : weights) {
        cin >> w;
        lo = max(lo, w);
        hi += w;
    }
    while (lo < hi) {
        long long mid = lo + (hi - lo) / 2;
        if (days_needed(weights, mid) <= d) hi = mid;
        else lo = mid + 1;
    }
    cout << lo << '\n';
}
return 0;
}

```

4.8 Python solution

```

import sys

def days_needed(weights: list[int], capacity: int) -> int:
    days = 1
    cur = 0
    for w in weights:
        if cur + w <= capacity:
            cur += w
        else:
            days += 1
            cur = w
    return days

def solve_case(weights: list[int], days: int) -> int:
    lo = max(weights)
    hi = sum(weights)
    while lo < hi:
        mid = (lo + hi) // 2
        if days_needed(weights, mid) <= days:
            hi = mid
        else:
            lo = mid + 1
    return lo

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []

```

```
for _ in range(t):
    n = data[idx]
    days = data[idx + 1]
    idx += 2
    weights = data[idx:idx + n]
    idx += n
    out.append(str(solve_case(weights, days)))
sys.stdout.write("\n".join(out))
```

4.9 Common wrong answers

- Setting `lo = 0`, then allowing capacities smaller than the heaviest package inside the predicate.
- Counting days incorrectly after starting a new day.
- Using `while lo + 1 < hi` but printing the wrong boundary for a minimum feasible search.
- Using `int` for the sum of weights in C++.

5 Stress testing strategy

For binary search problems, the random oracle can brute force the answer on small ranges:

- square threshold: compare with exact integer square root;
- maximum distance: try all combinations for small n ;
- minimum capacity: try every capacity from max weight to total sum.

The local generators do this. This is especially useful because binary-search bugs often pass samples and fail only at boundaries.

6 What to carry forward

Binary search on the answer is a pattern for turning optimization into decision. The hard part is usually not the loop; it is proving monotonicity and writing a correct feasibility check.

This pattern will reappear in greedy problems, graph bottleneck problems, flows, dynamic programming optimization, and geometry.

7 Problem D: Factory Deadline

7.1 Restatement and predicate

Machines run in parallel; machine i produces $\text{floor}(\text{minutes} / \text{time}[i])$ items by a proposed deadline. The question “can we make at least goal items by x ?” is monotone: if it is true at x , it stays true at every later minute.

7.2 Bounds, trace, and algorithm

Zero minutes is infeasible because the goal is positive. $\text{min}(\text{time}) * \text{goal}$ is feasible because the fastest machine can work alone; the local bounds keep it inside signed 64-bit range. With times 2 and 3, seven items need nine minutes: at eight minutes production is $4+2=6$, while at nine it is $4+3=7$.

```
lo = 0; hi = fastest_time * goal
while lo < hi:
    mid = lo + (hi-lo)/2
    if sum(mid / time[i]) >= goal: hi = mid
    else: lo = mid + 1
print lo
```

7.3 Correctness proof

For any x , each machine’s production is exactly $\text{floor}(x/\text{time}[i])$, so the predicate computes total production correctly. Increasing x cannot decrease any term, hence feasibility is monotone. The binary-search invariant keeps lo infeasible (unless it is the answer boundary) and hi feasible; each iteration retains the first feasible value in the interval. When they meet, that value is the minimum feasible deadline, which is the requested answer.

7.4 Complexity and implementation notes

Each predicate is $O(n)$, so the solution is $O(n \log(\text{min_time} * \text{goal}))$ time and $O(n)$ memory for the input. In C++, use `long long` and stop summing once total reaches goal to avoid unnecessary growth. Python integers are safe, but the same early break is useful. Test one machine, goal one, equal machine times, and a case near the 64-bit bound. The random tests use minute-by-minute brute force only on small instances plus a 200000-machine scale case.

7.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int test_count;
    cin >> test_count;

    while (test_count--) {
        int n;
        long long goal;
        cin >> n >> goal;

        vector<long long> time(n);
        for (long long& x : time) {
            cin >> x;
        }

        long long lo = 0;
        long long hi = *min_element(time.begin(), time.end()) * goal;

        while (lo < hi) {
            long long mid = lo + (hi - lo) / 2;
            long long made = 0;

            for (long long machine_time : time) {
                made += mid / machine_time;
                if (made >= goal) break;
            }

            if (made >= goal) {
                hi = mid;
            } else {
                lo = mid + 1;
            }
        }

        cout << lo << '\n';
    }
}
```

7.6 Full Python solution

```
import sys

def minimum_deadline(time: list[int], goal: int) -> int:
    lo = 0
    hi = min(time) * goal

    while lo < hi:
        mid = (lo + hi) // 2
        made = 0

        for machine_time in time:
            made += mid // machine_time
            if made >= goal:
                break

        if made >= goal:
            hi = mid
        else:
            lo = mid + 1

    return lo

data = list(map(int, sys.stdin.buffer.read().split()))
i = 1
out = []

for _ in range(data[0]):
    n = data[i]
    goal = data[i + 1]
    i += 2
    time = data[i:i + n]
    i += n
    out.append(str(minimum_deadline(time, goal)))

sys.stdout.write("\n".join(out))
```